Beyond Nominal Ratios: Language-Aware Prompt Compression for Multi-Hop Question Answering

Thanh-Nhan Truong[1,2(✉)], Ngoc-Vy Tran[1,2], and Nhat-Minh Tran[1,2]

[1] Faculty of Information Technology, University of Science, Ho Chi Minh City, Vietnam
{25C1505320, 25C1506947, 24C11055}@student.hcmus.edu.vn
[2] Vietnam National University, Ho Chi Minh City, Vietnam

Abstract. Prompt compression is reported as a ratio: the fraction of the context a compressor keeps. We show that this number is not comparable across languages, and that the gap is large enough to reverse which method is the cheaper one. Measuring token fertility on parallel WMT24++ sentences across four writing systems, the same content costs 4.97× more tokens in Hindi, 3.89× in Thai and 2.49× in Vietnamese than in English under the commercial tokenisers that bill. A compressor reporting the same nominal ratio in two languages is therefore not delivering the same savings. We propose an *effective ratio* that prices retention in English-equivalent tokens: it separates tokenisers a nominal ratio calls identical to within 1.02× but which differ by 1.80× in what they actually cost.

To ground this we reimplement IterCOMP, a recent training-free compressor evaluated only on English, and give it its first evaluation on Vietnamese multi-hop QA, released as IterCOMP-X. Across the complete VimQA benchmark and three English datasets ($n \geq 1{,}003$ each, read by Qwen2.5-7B at 4-bit), sentence-level compression beats token-level by +10.72 to +18.58 F1, though against the uncompressed baseline it wins on three of four. Two effects appear only outside English: a boolean-answer normalisation step worth +8.67 F1 in Vietnamese and exactly 0.0 in English, and non-diacritic text - ubiquitous online - costing every branch about 20 F1, with no measurable extra loss to the compressed branch.

Keywords: Prompt compression · Multi-hop question answering · Tokeniser fertility · Vietnamese NLP · Retrieval-augmented generation

1 Introduction

Multi-hop question answering aggregates evidence across documents, and retrieval-augmented generation [10] supplies those documents at two costs: input length grows with their number, and irrelevant passages degrade reader accuracy.

Prompt compression addresses both, but existing methods [7,11,17,24] compress once, conditioned on the original question, and so miss what defines multi-hop QA: the information needed at a later hop becomes identifiable only once the earlier hop's evidence is in. IterCOMP [28] embeds that reasoning into the compression loop: Fig. 1 follows one VimQA question through it, where the sentence holding the answer falls below the threshold on the first pass and is recovered on the second. The method ships no implementation, however, and is evaluated only on English.

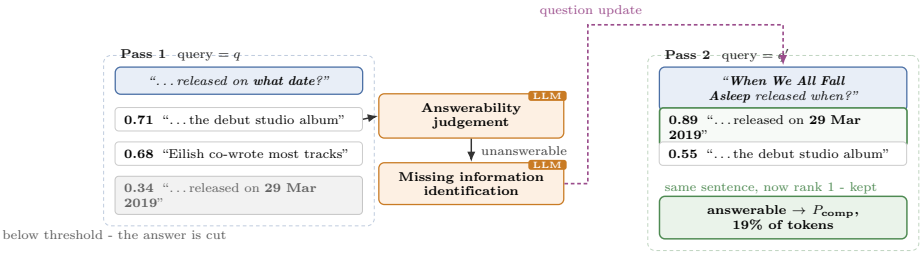


Fig. 1. One VimQA question through two IterCOMP passes. The sentence carrying the release date scores 0.34 and is filtered out, so a single-pass compressor loses the answer; re-scored against the follow-up q' it rises to 0.89 and is kept.

Four findings follow: one about compression ratios across languages, three about Vietnamese in particular.

- An effective ratio that prices retention in English-equivalent tokens. A nominal ratio means different things in different languages, understating Vietnamese by 2.39×, because the tokeniser that bills is not the one that counts.
- A boolean-answer normalisation step worth +8.67 F1 on Vietnamese (95% CI $[+6.98, +10.47]$) and exactly 0.0 on all three English datasets - a linguistic fix, not a metric-inflating one.
- Evidence that Vietnamese does not handicap the compressor: IterCOMP-X recovers 84% of the gold-evidence reference on VimQA, more than on any of the three English datasets (56–76%), so the Vietnamese setting is the easiest of the four rather than the hardest (Section 5.4).
- The first measurement of non-diacritic Vietnamese. Stripping diacritics costs about 20 F1, and costs gold and compressed context indistinguishable amounts (20.70 against 18.66, difference-in-differences CI $[-2.71, +6.79]$), so we find no extraction error the compressor adds (Table 6).

We release this configuration as IterCOMP-X. Our contribution is the measurement rather than the loop: a metric that makes retention comparable across languages, the first Vietnamese evaluation of a multi-hop compressor, and the reimplementation that made both possible, which the original paper does not provide.

2 Related Work

2.1 Prompt Compression

Surveys divide the field in two [12]. *Extractive* methods keep a subset of tokens or sentences: LLMLingua [7] drops tokens by perplexity; LongLLMLingua [8] adds query awareness; LLMLingua-2 [17] recasts this as token classification, faster and multilingual - the decisive property here; Selective-Context [11] removes redundant lexical units. *Abstractive* methods regenerate shorter content, notably RECOMP [24]. All compress in a single pass.

2.2 Iterative Retrieval

A parallel line of work treats multi-hop QA as a loop rather than a single pass. Self-Ask [20] has the model pose and answer its own follow-up questions; IRCoT [23] interleaves retrieval with chain-of-thought so that each reasoning step conditions the next query; ReAct [26] generalises the pattern to interleaved reasoning and acting; HippoRAG [5] instead indexes the corpus so that multi-hop associations are available without repeated retrieval. What these share is direction: each iteration adds evidence. IterCOMP runs the same kind of loop with the opposite objective, using the reasoning state to decide what to discard, so the context shrinks as the loop proceeds. All of this work is evaluated on English benchmarks.

2.3 Tokeniser Disparity

Subword tokenisers split non-English text more finely, so the same content costs more tokens. Ahia et al. [1] price this against commercial APIs, and Petrov et al. [18] frame it as unfairness between languages. That literature measures the cost of text, and where compression work controls for the disparity at all, it does so inside the evaluation protocol by matching budgets in the target tokeniser. What remains unproposed is a reported quantity carrying the correction, so that published ratios stay comparable across languages; Section 3.1 supplies one.

2.4 Vietnamese NLP

Vietnamese is well served at the model level, with the encoder PhoBERT [15] and the text-to-text ViT5 [19] both established, and VimQA [9] supplies a multi-hop benchmark in the HotpotQA format. Compression has begun to reach the language: XProvence [14] prunes context in over 100 languages and lists Vietnamese among its transfer targets on MKQA, but evaluates single-hop QA and reports no per-language retention. To our knowledge no compressor has been evaluated on Vietnamese *multi-hop* QA, where sentence-level supporting facts make an Oracle branch available, and none reports what a Vietnamese context costs under the tokeniser that bills.

3 Methodology

3.1 Tokeniser Fertility and the Effective Ratio

A compression ratio is reported as a fraction of the context kept, and read as a saving. That reading assumes the unit being counted costs the same everywhere. It does not: the tokeniser that bills splits some writing systems far more finely than English, so the same nominal ratio buys different savings in different languages. We measure the gap here and propose a correction.

Table 1. Token fertility over 930 parallel English-Vietnamese sentence pairs from WMT24++ [3]. All 95% confidence intervals are narrower than ±0.04. Fertility is the mean of per-pair token ratios (Eq. 2), not the ratio of the mean counts in columns two and three; the two differ by up to 0.03 here because pair lengths vary. Fertility depends strongly on the tokeniser: a factor of two separates `cl100k_base` from XLM-R. Both OpenAI tokenisers are measured with `tiktoken` [16]. [a]GPT-4, GPT-3.5-turbo; [b]GPT-4o. Which tokeniser bills depends on the served model, so exposure to fertility is set by that choice as much as by the language.

Tokeniser	Vi tok	En tok	Fertility
`cl100k_base`[a]	105.6	44.1	**2.39**×
`o200k_base`[b]	63.3	43.3	1.49×
XLM-R (used by bge-m3)	57.9	49.7	1.19×
`\w+` (unit of "ratio")	47.6	35.7	1.34×

Cross-lingual Measurement. Fertility on Vietnamese alone proves nothing: if every language were billed alike, the quantity would not be worth reporting. Measuring 400 parallel WMT24++ pairs per language shows it is not.

Every language exceeds 1.0 with non-overlapping intervals (Fig. 2), so the mismatch is systematic rather than a Vietnamese quirk; Vietnamese at 2.49× is the mild case: Hindi is penalised twice as heavily. The 2.49× here (400 pairs) and the 2.39× in Table 1 (930 pairs) are different samples of the same corpus.

Crucially, fertility depends on which tokeniser is measured. Compression ratios count with `\w+`, where fertility is only 1.34×, so comparing them across languages is close to fair. API cost is not: it is billed on the served model's tokeniser, 2.39× under `cl100k_base` against 1.49× under `o200k_base`, while bge-m3's XLM-R is 1.19×. On the 200 VimQA contexts here a context averages 1,110 whitespace-delimited syllables but 2,364 `cl100k_base` tokens, so our ratios understate billing 2×.

Definition. Write $|x|_{\mathcal{T}}$ for the number of tokens the billed tokeniser $\mathcal{T}$ assigns to a string x, and let D be the retrieved documents and P_{comp} the compressed prompt. The quantity compression papers report is

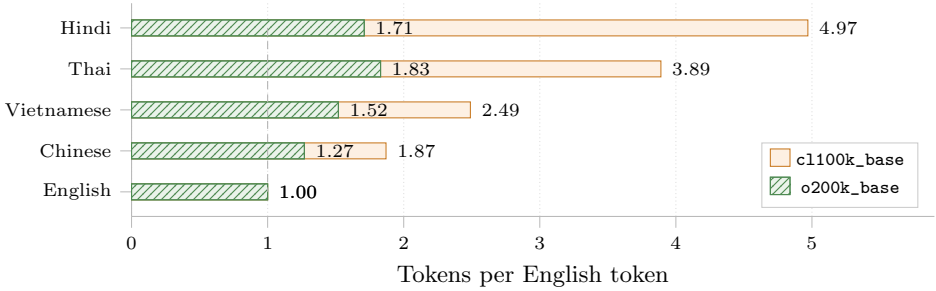


Fig. 2. What the same content costs in tokens, relative to its English translation, over 400 WMT24++ pairs per language. English is 1.0 by construction (dashed). Under `cl100k_base` a Hindi context costs almost five times its English counterpart, so two systems reporting the same nominal ratio in Hindi and in English are not delivering the same saving. Changing the tokeniser changes the tax more than changing the language does: `o200k_base` flattens every bar, and reorders Thai above Hindi.

$$\text{Ratio}_{\text{nom}} \;=\; \frac{|P_{\text{comp}}|_{\mathcal{T}}}{|D|_{\mathcal{T}}}. \tag{1}$$

Let $\phi_{\mathcal{T}}(\mathcal{L})$ be the fertility of language $\mathcal{L}$ under $\mathcal{T}$, measured over a parallel corpus $\mathcal{D}_{\text{par}}$ of aligned pairs as the mean of the per-pair ratios

$$\phi_{\mathcal{T}}(\mathcal{L}) \;=\; \frac{1}{|\mathcal{D}_{\text{par}}|} \sum_{(x_{\mathcal{L}},x_{\text{en}})\in\mathcal{D}_{\text{par}}} \frac{|x_{\mathcal{L}}|_{\mathcal{T}}}{|x_{\text{en}}|_{\mathcal{T}}}, \tag{2}$$

which is what Table 1 and Fig. 2 report. We define the *effective ratio* as what the compressed context costs against an English original of the same content,

$$\text{Ratio}_{\text{eff}} \;=\; \text{Ratio}_{\text{nom}} \cdot \phi_{\mathcal{T}}(\mathcal{L}), \tag{3}$$

which leaves English unchanged ($\phi = 1$) and rises exactly as far as the tokeniser penalises the language: keeping a fifth of a Vietnamese context is not the saving that keeping a fifth of an English one is. A single factor suffices because fertility belongs to the language-tokeniser pair, not to the compressor, so reporting both ratios costs one column and leaves the familiar number intact.

How Much a Nominal Ratio Conceals. The correction is a multiplication, so the effective ratio inherits the fertility spread by construction; what the following measures is the size of what a reported ratio hides. We hold the compressed contexts fixed - the same 50 VimQA questions, the same IterCOMP-X output, only the tokeniser changed (Table 2):

The contexts are byte-identical across the three rows, so three systems a nominal ratio calls identical to within 1.02× differ by 1.80× in what they actually cost - all of it the tokeniser's, none of it the compressor's. That is the argument for adding the second column rather than replacing the first.

Table 2. Nominal against effective ratio for one fixed set of compressed contexts (50 VimQA questions, same IterCOMP-X output, tokeniser varied). Bold marks the spread the two columns disagree on.

Tokeniser	nominal	effective
`cl100k_base`	0.236	0.564
`o200k_base`	0.240	0.357
`\w+`	0.234	0.314
spread	1.02×	**1.80**×

3.2 Iterative Multi-Hop Compression

Given a question q and document set D, we seek a compressed context P_{comp} with $|P_{\text{comp}}| \ll |D|$ such that the reader still answers correctly. IterCOMP is training-free: all parameters stay frozen. Each document is decomposed into sentence-level evidence segments $e_{i,j}$. Fig. 1 traced this loop on one question; we now give the two steps it turns on.

Evidence Filtering. The *semantic* signal is the inner product of dense representations from a text encoder $E(\cdot)$:

$$S_{\text{sem}}(q, e_{i,j}) = E(q)^\top E(e_{i,j}). \tag{4}$$

This can miss exact lexical cues. Following M3-Embedding [2], the importance weight of token t in a sequence $u \in \{q, e_{i,j}\}$ is

$$w_t^u = \text{ReLU}(\mathbf{w}_{\text{lex}}^\top E_t(u)), \tag{5}$$

where $\mathbf{w}_{\text{lex}}$ is a projection vector and $E_t(u)$ the encoder's contextual state at t. The *lexical* score is a sparse inner product over co-occurring tokens:

$$S_{\text{lex}}(q, e_{i,j}) = \sum_{t \in q \cap e_{i,j}} w_t^q \cdot w_t^{e_{i,j}}. \tag{6}$$

The two are combined into a dual-aspect relevance score:

$$S_{\text{dual}} = \lambda\, S_{\text{sem}} + (1 - \lambda)\, S_{\text{lex}}. \tag{7}$$

Filtering uses a percentile threshold rather than a fixed top-k. Writing $\mathcal{E} = \{e_{i,j}\}$ for the segments of every document in D and $\mathcal{S} = \{S_{\text{dual}}(q, e) \mid e \in \mathcal{E}\}$ for their scores,

$$E_{\text{cand}} = \{e \in \mathcal{E} \mid S_{\text{dual}}(q, e) \geq \text{Percentile}(\mathcal{S}, k)\}. \tag{8}$$

The threshold therefore adapts to each question's score distribution rather than committing to a fixed budget.

Answerability Judgement and Query Reformulation. These two steps let the query evolve with the evidence state instead of staying fixed at q. In step 2 the reader sees the evidence accumulated so far and is asked whether it suffices to answer q, replying with a single token. The reply is accepted on a substring test for `YES`, which separates the two intended labels but means a malformed reply containing that substring stops the loop early rather than costing an extra pass. When the evidence is judged insufficient, step 3 asks for one short follow-up question naming the information still missing, and that follow-up becomes q' for the next pass. The loop ends at whichever comes first: a sufficient verdict, `max_iter`, or a safety cap that stops the loop once the accumulated evidence exceeds 60% of the source - without it a reader that never reports sufficiency can return a "compressed" context longer than the original. Segments accumulate rather than replacing one another: from $E_{\text{accum}} \leftarrow \emptyset$, each pass m applies $E_{\text{accum}} \leftarrow E_{\text{accum}} \cup E_{\text{cand}}^{(m)}$, and a segment already chosen is not offered again. We emit P_{comp} in selection order rather than document order. Sorting the selected segments back into document order changes VimQA by -0.70 F1, CI $[-2.67, +1.22]$, so the reader is indifferent to it at this scale and we keep the simpler behaviour. Neither prompt is published in the original paper; ours are released in full, in Vietnamese and English.

4 Experimental Setup

4.1 Datasets

The three English datasets are standard for multi-hop QA and are exactly those used by the original paper. VimQA [9] is a Vietnamese multi-hop QA dataset built from Vietnamese Wikipedia, annotated with reasoning type, though the validation split is labelled `bridge` throughout despite the age-comparison template of Section 5.6 and gold supporting sentences; crucially, it uses the same format as HotpotQA, so our code runs on Vietnamese essentially unchanged. We use the distractor setting for all four, so no Wikipedia index is required. MuSiQue withholds test answers, so we report on development sets throughout, and every dataset runs under a single reader, Qwen2.5-7B [21] at 4-bit (Table 3).

Table 3. Development-set sizes. #Q is the full split and #eval the sample actually run; VimQA is evaluated in full.

Dataset	Lang.	#Q	#eval	#psg
HotpotQA [25]	En	7,405	1,500	10
2Wiki [6]	En	12,576	1,500	10
MuSiQue [22]	En	2,417	1,500	20
VimQA [9]	**Vi**	**1,003**	**1,003**	**10**

We adopt the original paper's hyperparameters (Table 4).

Table 4. Hyperparameters from the original paper, which used a LLaMA-3-8B [4] reader; VimQA, absent there, uses $k = 90$ as HotpotQA. The reader is 4-bit NF4 (bitsandbytes, double quantisation) under HuggingFace Transformers, decoding greedily at most 48 new tokens, so no sampling variance enters the numbers.

Parameter	Value
Max iterations	5
Evidence cap	60% of source
λ	0.6
k (percentile)	90; 85 for 2Wiki
Encoder	bge-m3 [2] (frozen)
Reader	Qwen2.5-7B-4bit

4.2 Baselines

Raw Documents concatenates everything with no compression. Oracle keeps only the gold supporting sentences. It is the reference condition for selection quality - not a per-question bound, since Raw still beats it on 7 to 10% of questions - and a diagnostic: if Oracle is low, the bottleneck is the reader rather than the compressor. LLMLingua-2 [17] is our main compression baseline, chosen because its token classifier is natively multilingual and covers Vietnamese, so it meets Vietnamese on equal terms and can run on all four datasets.

Six more run on VimQA at $n = 200$: LLMLingua [7], LongLLMLingua [8], Selective-Context [11], RECOMP [24] in its extractive and abstractive forms, and CompAct [27]. Each carries an English-only component, so they widen the field rather than compare like with like, and stay out of the main table. R2C (Retrieve-Read-Compress), the original paper's second-strongest baseline, has no public checkpoint.

4.3 Metrics

We compute Exact Match and token-level F1 under the standard HotpotQA normalisation, plus the nominal compression ratio. Tables report F1, the less brittle of the two on free-form answers; EM follows the same ordering and is in the released per-question records. Nominal retention is counted in `\w+` units, the unit the original protocol uses - words in English, syllables in Vietnamese, so the 1.34× figure is a lower bound on word-level fertility; Eq. 3 converts it to billed cost. That normalisation is built for English, so for Vietnamese we preserve diacritics: stripping them collapses distinct syllables and renders EM/F1 meaningless.

5 Results and Discussion

5.1 Sentence-Level against Token-Level Compression

Fig. 3 plots accuracy against context retained; sentence-level compression beats token-level LLMLingua-2 by +10.72 to +18.58 F1 (Table 5, last row), every

Table 5. Main results: F1* - F1 with the question-guided boolean normalisation of Section 5.4, which is inert on English - under Qwen2.5-7B at 4-bit; the second row gives n. The last two rows are IterCOMP-X minus that baseline. † paired bootstrap interval excludes zero. Bold marks whichever of Raw and IterCOMP-X is higher. Per-dataset discussion is in the released repository.

	VimQA	MuSiQue	2Wiki	HotpotQA
n	1003	1500	1500	1500
Oracle	61.03	44.90	57.65	71.40
Raw	46.87	21.16	35.72	**59.24**
IterCOMP-X	**51.29**	**25.35**	**36.72**	54.58
LLML-2	35.31	14.63	19.24	36.00
Δ vs. Raw	$+4.42^{\dagger}$	$+4.19^{\dagger}$	$+1.00$	$-4.66^{\dagger}$
Δ vs. LLML-2	$+15.98^{\dagger}$	$+10.72^{\dagger}$	$+17.48^{\dagger}$	$+18.58^{\dagger}$

paired bootstrap interval clear of zero, while keeping 0.194 to 0.354 of the context. The two are not matched on cost: LLMLingua-2 classifies tokens once with a small encoder, IterCOMP-X calls a 7B model twice per pass, 2.6 to 7.4× the time per question. The margin shows that sentence-level selection keeps multi-hop evidence which token-level scoring discards; it does not show that the method is cheaper. This is the one result independent of dataset, reader and sample size. At $n = 200$ on VimQA it also leads every one of them, by +11.36 F1 over LLMLingua-2, CI $[+4.93, +17.84]$.

5.2 Reproduction of the Reported Ordering

The paper reports Oracle > IterCOMP > Raw > LLMLingua-2. We reproduce that ordering on VimQA, where IterCOMP-X leads Raw by +4.42 F1 with the interval clear of zero, and on MuSiQue at +4.19. The MuSiQue margin read only +3.31 at $n = 500$ and crossed zero, so tripling the sample is what decided it. On 2Wiki the order matches but +1.00 does not separate, and on HotpotQA it inverts: Raw overtakes IterCOMP-X by 4.66 F1, again with the interval clear. The four gaps are therefore +4.42, +4.19, +1.00 and −4.66 F1, and Section 5.3 accounts for the reversal.

At $n = 50$ those gaps sat inside the range this design can resolve, so we held the accuracy claim until the full runs settled it. The interpretation that the margin grows with task complexity does not survive either: +15.98 on VimQA against +10.72 on MuSiQue.

5.3 Reader Dependence of the Ordering

Oracle exceeds Raw on every dataset while reading 5.7% to 16.4% of the tokens, confirming the premise the literature rests on: irrelevant passages do degrade accuracy. The size of that gap decides whether compression can exploit it. Oracle/Raw is 2.12 on MuSiQue, 1.61 on 2Wiki and 1.30 on VimQA, the three

where IterCOMP-X holds its own. On HotpotQA it is only 1.21, and Raw wins outright. The premise is therefore reader-dependent: a reader that already copes with distractors leaves compression little to recover. Replacing the reader with Qwen2.5-0.5B on HotpotQA ($n = 500$) moves IterCOMP-X from -4.66 to $+7.66$ F1 against Raw, the ordering the original paper reports with its own weaker reader.

That swap alone does not isolate the reader, because the original design uses the reader as its own answerability judge, so the smaller model both reads and compresses - and it compresses far less, retaining 0.532 of the context against 0.192. We therefore ran the control: judge pinned at 7B, reader at 0.5B. The compressor returns to 0.192 retention, identical to the 7B configuration, and IterCOMP-X still leads Raw by +5.70 F1, CI $[+1.90, +9.63]$. Holding compression fixed and changing only the reader reproduces the reversal, so it belongs to the reader rather than to the compressor. The remaining +1.96 F1 between the two 0.5B runs is what the weaker judge's extra context buys.

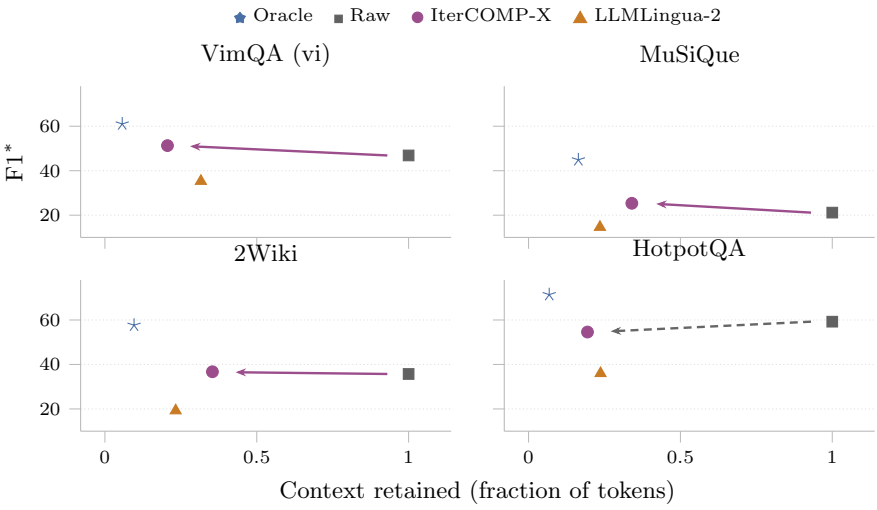


Fig. 3. What compression buys, one panel per dataset. The horizontal axis is how much context a method keeps, the vertical axis how accurately the reader answers from it, so a method is better the further up and to the left it sits. The arrow runs from Raw, which keeps everything, to IterCOMP-X: pointing up-left means the compressed context is both smaller *and* more accurate. That happens on VimQA and MuSiQue; on 2Wiki the arrow is flat (+1.00, not separating); on HotpotQA it points down (dashed), so there only the token saving survives. LLMLingua-2 sits below IterCOMP-X in every panel at comparable retention, and Oracle marks what the reader reaches when handed the gold evidence.

5.4 Vietnamese-Specific Results

Four results appear only on Vietnamese. Boolean answer normalisation is worth +8.67 F1 on VimQA at $n = 1{,}003$, interval $[+6.98, +10.47]$, and exactly 0.0 on each of MuSiQue, 2Wiki and HotpotQA, isolating a language-specific fix from a metric-inflating one: a reader asked a Vietnamese question often answers "Yes", which is correct and scores 0 against the gold "đúng". The step post-processes the reader's output and is gated by the *question* - it fires on the closing *phải không / đúng không* frame, never on the gold answer, so it is available at inference and cannot turn a wrong answer right: a wrong-polarity prediction maps to the opposite label and still scores zero. Vietnamese does not handicap the compressor: IterCOMP-X recovers 84% of the gold-evidence reference on VimQA (51.29 against 61.03), the highest of the four, against 76% on HotpotQA, 64% on 2Wiki and 56% on MuSiQue. The Vietnamese dataset is the easiest to recover on, not the hardest. We do not claim to explain the spread: MuSiQue's twenty candidate passages against ten elsewhere fit its position at the bottom, but the three ten-passage datasets themselves range over twenty points.

5.5 Diacritic Removal

Vietnamese is often typed without diacritics online, a setting no compression paper evaluates, and stripping them is expensive. Diacritics carry tone (*ma*, *mà*, *má*, *mả*, *mã*, *mạ* collapse to one form) and segmental contrasts (ă â ê ô ơ ư, đ), so *đang* and *dáng* merge too; across VimQA this shrinks the syllable-type inventory by 21.4%, and both branches lose about 20 F1. Gold context leads throughout, by 10.52 F1 with diacritics and by 8.48 and 6.75 without, every interval clear of zero at $n = 500$ where none separated at $n = 200$. What stripping does *not* do is single out the compressor: gold loses 20.70 F1 and the compressed branch 18.66, a difference-in-differences of +2.04, CI $[-2.71, +6.79]$. We attribute the loss to the reader failing on merged syllables, though the interval leaves room for a small effect either way.

One point matters for anyone repeating this: supporting facts address documents by title, so diacritics must come off the annotations as well as the context. Strip one side only and the match breaks, leaving the gold branch empty.

5.6 Error Analysis

We hand-categorised 50 IterCOMP-X predictions on VimQA. Most errors belong to the reader: Oracle sees the gold evidence yet reaches only 61.03 F1, so 80% of IterCOMP-X's 48.7 F1 of error survives perfect retrieval. One error in ten is not a compression failure at all: a yes/no question answered correctly in substance but in the wrong surface form, scoring F1 of 0. A smaller reader does this far more often on the identical prompt - "tức là yes" for "đúng", the language confusion small models are prone to [13] - so boolean answers need language-aware normalisation rather than a compression fix. The rest give a related but wrong entity, or are compositional.

Table 6. Non-diacritic Vietnamese ($n = 500$, 7B reader). Gold context leads in every setting and † every paired bootstrap interval clears zero: $[-14.40, -6.65]$, $[-12.25, -4.77]$ and $[-10.57, -2.94]$. None separated at $n = 200$, so the gap is real but needed the larger sample. Stripping costs both branches about 19-21 F1. `strip` leaves the gold answer accented, so the reader must restore marks it never saw; `both` strips the reference too, scoring entity recovery rather than diacritics.

Setting	IterCOMP-X	Gold	Δ
`keep`	47.47	58.00	$-10.52^{\dagger}$
`strip`	28.81	37.29	$-8.48^{\dagger}$
`both`	30.61	37.36	$-6.75^{\dagger}$

A further 9.8% of VimQA is a single question template - *"X lớn tuổi hơn Y phải không?"*, "is X older than Y?", and its negation *nhỏ tuổi hơn* - on which 44.9% of answers score zero against 37.1% elsewhere, because the task is arithmetic over two birth dates rather than retrieval; under a 0.5B reader it collapses to 92.1% zeros.

6 Conclusion

A compression ratio is not comparable across languages, and a nominal figure conceals a 1.80× spread in true cost across tokenisers it reports as identical to within 1.02×; the effective ratio is one extra column and fixes that. Reproducing IterCOMP without its code and giving it its first Vietnamese evaluation, released as IterCOMP-X, shows sentence-level beating token-level compression on all four datasets while winning on three of four against the uncompressed baseline. Neither Vietnamese nor the loss of its diacritics handicaps the compressor; the reader does, failing on ∼30% of questions even given gold evidence.

Three limits bound these claims. Every headline number comes from a single reader; the one control we ran against that, pinning the judge at 7B while the reader shrinks (Section 5.3), settles the HotpotQA reversal but tests two reader sizes rather than a range. Fertility is measured on four writing systems while compression runs on two, so the effective ratio is argued generally and shown on one language pair; a Hindi or Thai multi-hop set with sentence-level supporting facts would test it where fertility is worst rather than mildest, and none exists today. Absolute numbers are not comparable to the original paper's, which released no prompts and used a different reader at a different operating point (0.340 retention against 0.14).

Acknowledgments. Supported by the Faculty of Information Technology, University of Science, VNU-HCM.

References

1. Ahia, O., Kumar, S., Gonen, H., Kasai, J., Mortensen, D., Smith, N.A., Tsvetkov, Y.: Do all languages cost the same? Tokenization in the era of commercial language models. In: Proceedings of EMNLP (2023), https://aclanthology.org/2023.emnlp-main.614/
2. Chen, J., Xiao, S., Zhang, P., Luo, K., Lian, D., Liu, Z.: M3-Embedding: Multilinguality, multi-functionality, multi-granularity text embeddings through self-knowledge distillation. In: Findings of ACL (2024), https://aclanthology.org/2024.findings-acl.137/
3. Deutsch, D., Briakou, E., Caswell, I., Finkelstein, M., Galilee, R., Juraska, J., Kovacs, G., Lee, A., Rudder, J., Freitag, M.: WMT24++: Expanding the language coverage of WMT24 to 55 languages and dialects. In: Proceedings of WMT (2025), https://huggingface.co/datasets/google/wmt24pp
4. Grattafiori, A., Dubey, A., Jauhri, A., et al.: The Llama 3 herd of models. Technical report, Meta AI (2024)
5. Gutiérrez, B.J., Shu, Y., Gu, Y., Yasunaga, M., Su, Y.: HippoRAG: Neurobiologically inspired long-term memory for large language models. In: Proceedings of NeurIPS (2024)
6. Ho, X., Duong Nguyen, A.K., Sugawara, S., Aizawa, A.: Constructing a multi-hop QA dataset for comprehensive evaluation of reasoning steps. In: Proceedings of COLING (2020), https://aclanthology.org/2020.coling-main.580/
7. Jiang, H., Wu, Q., Lin, C.Y., Yang, Y., Qiu, L.: LLMLingua: Compressing prompts for accelerated inference of large language models. In: Proceedings of EMNLP (2023), https://aclanthology.org/2023.emnlp-main.825/
8. Jiang, H., Wu, Q., Luo, X., Li, D., Lin, C.Y., Yang, Y., Qiu, L.: LongLLMLingua: Accelerating and enhancing llms in long context scenarios via prompt compression. In: Proceedings of ACL (2024), https://aclanthology.org/2024.acl-long.91/
9. Le, K., Nguyen, H., Le Thanh, T., Nguyen, M.: VIMQA: A Vietnamese dataset for advanced reasoning and explainable multi-hop question answering. In: Proceedings of LREC (2022), https://aclanthology.org/2022.lrec-1.700/
10. Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.t., Rocktäschel, T., Riedel, S., Kiela, D.: Retrieval-augmented generation for knowledge-intensive NLP tasks. In: Proceedings of NeurIPS (2020)
11. Li, Y., Dong, B., Guerin, F., Lin, C.: Compressing context to enhance inference efficiency of large language models. In: Proceedings of EMNLP (2023), https://aclanthology.org/2023.emnlp-main.391/
12. Li, Z., Liu, Y., Su, Y., Collier, N.: Prompt compression for large language models: A survey. In: Proceedings of NAACL (2025), https://aclanthology.org/2025.naacl-long.368/
13. Marchisio, K., Ko, W.Y., Bérard, A., Dehaze, T., Ruder, S.: Understanding and mitigating language confusion in LLMs. In: Proceedings of EMNLP (2024), https://aclanthology.org/2024.emnlp-main.380/
14. Mohamed, Y., Elhoseiny, M., Formal, T., Chirkova, N.: XProvence: Zero-Cost Multilingual Context Pruning for Retrieval-Augmented Generation. In: Proceedings of ECIR. Lecture Notes in Computer Science, vol. 16484. Springer (2026)
15. Nguyen, D.Q., Nguyen, A.T.: PhoBERT: Pre-trained language models for Vietnamese. In: Findings of EMNLP (2020), https://aclanthology.org/2020.findings-emnlp.92/

16. OpenAI: tiktoken: A fast BPE tokeniser for use with OpenAI's models. https://github.com/openai/tiktoken (2024)
17. Pan, Z., Wu, Q., Jiang, H., Xia, M., Luo, X., Zhang, J., Lin, Q., Rühle, V., Yang, Y., Lin, C.Y., Zhao, H.V., Qiu, L., Zhang, D.: LLMLingua-2: Data distillation for efficient and faithful task-agnostic prompt compression. In: Findings of ACL (2024), https://aclanthology.org/2024.findings-acl.57/
18. Petrov, A., La Malfa, E., Torr, P., Bibi, A.: Language model tokenizers introduce unfairness between languages. In: Advances in Neural Information Processing Systems (NeurIPS) (2023)
19. Phan, L., Tran, H., Nguyen, H., Trinh, T.H.: ViT5: Pretrained text-to-text transformer for Vietnamese language generation. In: Proceedings of NAACL Student Research Workshop (2022), https://aclanthology.org/2022.naacl-srw.18/
20. Press, O., Zhang, M., Min, S., Schmidt, L., Smith, N.A., Lewis, M.: Measuring and narrowing the compositionality gap in language models. In: Findings of EMNLP (2023), https://aclanthology.org/2023.findings-emnlp.378/
21. Qwen Team: Qwen2.5 technical report. Technical report, Alibaba Group (2025)
22. Trivedi, H., Balasubramanian, N., Khot, T., Sabharwal, A.: MuSiQue: Multihop questions via single-hop question composition. Transactions of the Association for Computational Linguistics **10** (2022), https://aclanthology.org/2022.tacl-1.31/
23. Trivedi, H., Balasubramanian, N., Khot, T., Sabharwal, A.: Interleaving retrieval with chain-of-thought reasoning for knowledge-intensive multi-step questions. In: Proceedings of ACL (2023), https://aclanthology.org/2023.acl-long.557/
24. Xu, F., Shi, W., Choi, E.: RECOMP: Improving retrieval-augmented lms with context compression and selective augmentation. In: Proceedings of ICLR (2024), https://openreview.net/forum?id=mlJLVigNHp
25. Yang, Z., Qi, P., Zhang, S., Bengio, Y., Cohen, W.W., Salakhutdinov, R., Manning, C.D.: HotpotQA: A dataset for diverse, explainable multi-hop question answering. In: Proceedings of EMNLP (2018), https://aclanthology.org/D18-1259/
26. Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., Cao, Y.: ReAct: Synergizing reasoning and acting in language models. In: Proceedings of ICLR (2023)
27. Yoon, C., Lee, T., Hwang, H., Jeong, M., Kang, J.: CompAct: Compressing retrieved documents actively for question answering. In: Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (2024)
28. Yun, J., Kim, Y.: IterCOMP: Reasoning-aware adaptive prompt compression for multi-hop question answering. In: Proceedings of ACL (Volume 1: Long Papers). San Diego, California (2026), https://aclanthology.org/2026.acl-long.1559/